

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA1402

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:100

Annexure A

CASE STUDY

Code of Conduct:

I N.SANDIYANITTIANANDAM (192571029) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q. No	Understanding of Concepts (25)	Experimental Design (25)	Use of Tools (20)	Analysis of Results (20)	Documentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

Annexure A

CASE STUDY

Set 1

Q1. Both parse trees and syntax trees are used to represent program structure, but differ significantly in complexity and abstraction level. Compare these representations in terms of memory usage, ease of traversal, suitability for optimization, and their role in different phases of compilation such as semantic analysis and code generation. These require careful consideration of attribute dependency graphs to ensure correct evaluation order without conflicts. Furthermore, representation choice affects compiler performance. Finally, selecting the right abstraction improves overall efficiency.

SOLUTION

Comparison of Parse Trees and Syntax Trees

Both **parse trees** and **syntax trees (Abstract Syntax Trees/ASTs)** represent the structure of a program, but they differ in detail and purpose.

Aspect	Parse Tree	Syntax Tree (AST)
Structure	Shows the complete grammatical structure of the source program	Shows only the essential syntactic structure
Memory Usage	High, because it contains grammar rules, terminals, non-terminals, and punctuation	Lower, because unnecessary grammar details are removed
Traversal	More complex and time-consuming due to many nodes	Easier and faster because the tree is compact
Semantic Analysis	Can be used, but contains more information than necessary	Highly suitable for type checking, scope checking, and attribute evaluation
Optimization	Less convenient because of redundant nodes	Very suitable for optimization and transformations
Code Generation	Usually not directly used	Commonly used as an intermediate representation for generating code
Compiler Phase	Mainly associated with syntax analysis/parsing	Mainly used in semantic analysis, optimization, and code generation

Example

For the expression:

$a + b * c$

A parse tree represents all grammar productions used to derive the expression, including intermediate grammar symbols. An AST simplifies this into:

```
+  
→ a  
→ *  
  → b  
  → c
```

Thus, the AST removes unnecessary syntactic details while preserving the meaning of the expression.

Attribute Dependency and Evaluation

During **semantic analysis**, attributes such as data types and values may be associated with tree nodes. An **attribute dependency graph** determines the order in which these attributes must be evaluated. A correct evaluation order prevents circular dependencies and conflicts.

Role in Compiler Efficiency

Parse trees are useful for verifying that the source program follows the grammar, but their large size increases memory consumption and traversal time. ASTs provide a more compact abstraction, making semantic analysis, optimization, and code generation more efficient.

Q2. A compiler designer must choose between bottom-up and top-down evaluation strategies for implementing syntax-directed definitions. Justify this choice by analyzing grammar restrictions, attribute dependencies, parsing efficiency, and the ease of integrating semantic actions within each approach. These require careful consideration of attribute dependency graphs to ensure correct evaluation order without conflicts. Furthermore, evaluation strategy influences implementation complexity. Finally, choosing the right method ensures better performance.

SOLUTION

Bottom-Up vs Top-Down Evaluation of Syntax-Directed Definitions

A compiler designer should choose the evaluation strategy based on the **grammar, attribute dependencies, parsing method, and implementation requirements**.

Factor	Bottom-Up Evaluation	Top-Down Evaluation
Grammar Restrictions	Works well with S-attributed definitions and LR grammars	Best suited for L-attributed definitions and LL grammars

Attribute Dependencies	Synthesized attributes can be evaluated naturally after processing children	Inherited attributes can be evaluated before processing a child
Parsing Efficiency	Efficient with LR parsing and handles a larger class of grammars	Simple and efficient for suitable LL grammars
Semantic Actions	Actions are generally performed when reductions occur	Actions can be placed directly within productions and executed during expansion
Implementation	More complex because semantic actions are associated with reductions	Easier to understand and implement for simple grammars
Usefulness	Suitable for bottom-up parsers and compiler phases requiring synthesized information	Suitable when information must flow from parent or left sibling to a child

Attribute Dependency Consideration

An **attribute dependency graph** must be examined before selecting the evaluation strategy. If an attribute depends only on child attributes, bottom-up evaluation is natural because children are processed first. If a child requires information from its parent or a previously processed sibling, top-down evaluation is generally more appropriate.

Justification

Bottom-up evaluation is preferred when the grammar naturally supports synthesized attributes and an LR parser is being used. It provides efficient evaluation after reductions and can handle complex grammars.

Top-down evaluation is preferable when inherited attributes are important and the grammar is suitable for LL parsing. Semantic actions can be integrated during production expansion, making the implementation relatively straightforward.

Q3. In complex arithmetic and logical expressions involving multiple operators, precedence rules, and associativity constraints, attribute dependencies may create conflicts in evaluation order. Analyze how these dependencies affect semantic rule execution and propose techniques such as dependency graphs or evaluation scheduling to resolve them. These require careful consideration of attribute dependency graphs to ensure correct evaluation order without conflicts. Furthermore, resolving conflicts is critical for correctness. Finally, structured evaluation improves reliability.

SOLUTION

Attribute Dependencies in Complex Expressions

In complex arithmetic and logical expressions, **operator precedence, associativity, and attribute dependencies** determine the correct order in which semantic rules must be executed. If these dependencies are not handled correctly, the compiler may produce incorrect intermediate results or generated code.

1. Effect of Attribute Dependencies

Consider:

$a + b * c$

Because $*$ has higher precedence than $+$, the compiler must evaluate $b * c$ before adding a . Similarly, for:

$a - b - c$

left associativity requires:

$(a - b) - c$

If semantic rules are executed in the wrong order, the resulting value or type information may be incorrect.

Dependencies can occur between:

- **Synthesized attributes** — depend on child nodes.
- **Inherited attributes** — depend on the parent or neighboring nodes.
- **Type attributes** — may need to be determined before generating operations.
- **Temporary/intermediate values** — depend on previously evaluated subexpressions.

2. Dependency Graph

A **dependency graph** represents attributes as nodes and their dependencies as directed edges.

For $a + b * c$:

$b, c \rightarrow (b * c) \rightarrow a + (b * c)$

This clearly indicates that multiplication must be processed before addition.

The compiler can detect **circular dependencies** in the graph. If a cycle exists, the attributes cannot be evaluated using a simple ordering and the semantic rules must be redesigned.

3. Evaluation Scheduling

After constructing the dependency graph, the compiler can use **topological sorting** to determine a valid evaluation order.

For example:

1. Evaluate b
2. Evaluate c
3. Evaluate $b * c$
4. Evaluate a
5. Evaluate $a + (b * c)$

This ensures that every attribute is computed only after the information it depends on is available.

4. Techniques to Resolve Conflicts

- **Use dependency graphs** to explicitly identify attribute relationships.
- **Apply topological sorting** to obtain a valid evaluation sequence.
- **Separate inherited and synthesized attributes** according to their direction of information flow.
- **Use precedence and associativity rules** while constructing the syntax/AST.
- **Detect circular dependencies** before semantic evaluation.
- **Use intermediate attributes or temporary nodes** when a direct dependency creates an ordering problem.

Q4. A statically typed programming language enforces type checking during compilation to ensure correctness. Evaluate how such a type system helps detect errors early, prevents invalid operations, and improves software reliability, while also discussing any limitations in flexibility compared to dynamically typed systems. These require careful consideration of attribute dependency graphs to ensure correct evaluation order without conflicts. Furthermore, strong typing enhances safety. Finally, balancing flexibility and strictness is important.

SOLUTION

Evaluation of a Statically Typed System

A **statically typed programming language** performs type checking primarily during compilation. Each variable, expression, and function is associated with a type, and the compiler verifies that operations are valid before the program runs.

1. Early Error Detection

Static typing detects errors during compilation rather than during execution. For example:

```
int age = "twenty";
```

The compiler identifies the type mismatch before the program is executed. This reduces runtime failures.

2. Prevention of Invalid Operations

The compiler prevents operations that are incompatible with the declared types. For example, attempting to multiply a string directly by an object that does not support such an operation can be rejected during compilation.

3. Improved Software Reliability

Static type checking:

- Reduces runtime type-related errors.
- Makes program behavior more predictable.
- Helps developers identify mistakes early.
- Improves maintainability in large software projects.
- Allows compilers to perform certain optimizations safely.

4. Better Tool Support

Because types are known during compilation, development tools can provide better **code completion, refactoring, navigation, and error detection**.

5. Limitations Compared with Dynamic Typing

The main disadvantage is **reduced flexibility**. Programmers must often explicitly declare or manage types, and changing data representations may require additional code.

For example, dynamically typed languages allow a variable to hold different types at different times:

```
x = 10  
x = "Hello"
```

A statically typed language may reject such reassignment unless the type system explicitly permits it.

Static typing can therefore introduce:

- More code or type declarations.
- Longer compilation or type-checking processes.
- Less flexibility for rapidly changing data.
- Additional complexity when working with highly dynamic structures.

Q5. A modern programming language supports advanced features such as polymorphism, function overloading, and type inference. Analyze the challenges faced by the compiler's type system in resolving types accurately, managing ambiguity, and maintaining efficient compilation performance. These require careful consideration of attribute dependency graphs to ensure correct evaluation order without conflicts. Furthermore, advanced features increase complexity. Finally, efficient resolution mechanisms are essential.

SOLUTION

Challenges in Advanced Type Systems

Modern programming languages provide features such as **polymorphism, function overloading, and type inference**, which improve flexibility and programmer productivity. However, these features make the compiler's type system more complex.

1. Type Resolution

The compiler must determine the correct type of expressions and variables, sometimes without explicit declarations. With type inference, the compiler may need to collect constraints from multiple expressions before determining a final type.

For example:

```
x = 10
y = x + 2.5
```

The compiler must infer compatible types for `x` and `y` and determine the appropriate arithmetic operation.

2. Handling Polymorphism

Polymorphism allows the same function or operation to work with different types. The compiler must determine which implementation or type instance is appropriate.

For example:

```
max(10, 20)
max(2.5, 5.5)
```

The compiler must resolve the appropriate version or generic instance without introducing type errors.

3. Function Overloading and Ambiguity

When multiple functions have the same name but different parameter types, the compiler must select the best matching function.

For example:


```
display(int)
display(float)
```

For a call such as `display(10)`, the compiler selects the integer version. However, implicit conversions can create ambiguity when multiple versions are equally suitable.

4. Attribute Dependencies

Type attributes can depend on information from several nodes in the syntax tree. The compiler therefore needs an appropriate **attribute dependency graph** and evaluation order. Circular or conflicting dependencies can make type resolution difficult.

5. Compilation Performance

Advanced type systems can increase compilation time because the compiler may need to:

- Generate and solve many type constraints.
- Search among overloaded functions.
- Perform generic type substitution.
- Check inheritance and polymorphic relationships.
- Repeatedly propagate inferred type information.

Techniques for Efficient Resolution

Compilers can improve type resolution using:

- **Constraint-based type inference** to represent type relationships systematically.
- **Dependency graphs** to establish a valid evaluation order.
- **Symbol tables** for efficient lookup of declarations and types.
- **Overload-resolution algorithms** to rank possible function matches.
- **Caching and memoization** to avoid repeated type calculations.
- **Early error detection** to eliminate invalid alternatives quickly.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand